Museum Guide project

Oral study guide: every step, every number, every likely question

MSA-DATB09-03 Neural Networks and Deep Learning Foundations, final project

Benjamin Caujolle, 20 October 2026

The answer in one sentence. A pretrained ResNet18 with a linear head recognises the style of a painting with 83.0 % accuracy on 220 new paintings, against 67.3 % for the best classical model and 41.4 % for our MLP, in 53 ms per photo on one CPU thread.

Contents

1. The pitch in 60 seconds 4
2. The basic ideas, explained simply 5
   - A neuron, weights and a network 5
   - The loss: how wrong we are 5
   - Gradient descent and the learning rate 5
   - Backpropagation and the 5 lines of the training loop 5
   - Epoch and mini-batch 5
   - Train, validation, test 6
   - Overfitting 6
   - ReLU, the activation 6
   - Initialisation and the first loss 6
   - Standardisation of the pixels 6
   - MLP versus CNN: why the MLP cannot see shapes 6
   - Convolution, filters and channels (session 7 slide) 6
   - A pretrained network and transfer learning 7
   - Frozen features (linear probe) versus fine-tuning 7
   - The tools against overfitting 7
   - Vanishing gradient 7
   - Data leakage 7
   - Confidence interval and bootstrap 7
   - Latency and CPU threads 8
3. Step 1: the data and a fair split (notebook 01) 9
   - What we downloaded 9
   - The split (make_split in data.py) 9
   - What we found in the data 9
4. Step 2: the MLP and its two checks (notebook 02) 10
   - The model 10
   - Check 1: the first loss 10
   - Check 2: overfit one batch 10
   - Baseline and image size 10
5. Step 3: monitoring and regularisation (notebook 03) 12
   - What we monitored 12
   - Learning-rate sweep (Adam, 8 values, 15 epochs each) 12
   - The tools against overfitting (lr 0.0001, 3 seeds each) 12
6. Step 4: the training doctor (notebook 04) 14
   - Lessons to say at the oral 15
7. Step 5: pretrained ResNet18 (notebook 05) 16
   - Why a CNN, and why pretrained 16
   - Method A: frozen features + head 16

Method B: fine-tuning layer4 16
Two silent bugs, measured 16
Painter leakage, measured fairly 17
8. Step 6: final test, CPU cost and recommendation (notebook 06) 18
Classical models (settings chosen on validation) 18
The final test, measured once 18
Are the gaps real? Paired bootstrap (same 220 paintings, 5 000 redraws) 19
Frozen or fine-tuned? Validation and test disagree 19
Speed and size 19
Recommendation to the director 19
9. Key numbers to know by heart 20
10. Concepts in simple words 21
11. Questions you may get, with answers 23
About the data 23
About the MLP 23
About training and diagnosis 23
About ResNet 24
About the results and the recommendation 24
About the project 24
12. Limits to admit, and next steps 25
13. Git history and how to reproduce 26
14. The 10-minute talk, slide by slide 27

1. The pitch in 60 seconds

Say this first, in this order: it answers the director's question in one breath.

1. **The question.** A museum wants a kiosk that recognises the style of a painting from a photo. The kiosk CPU is small and the team is small. Is a neural network worth it, or is a classical model enough?
2. **The data.** 1 500 WikiArt paintings, 5 styles (Baroque, Cubism, Impressionism, Pop Art, Ukiyo-e), 300 each. We split them so that **a known painter is never in two parts**: the scores measure the style, not the painter.
3. **Our own network (MLP)**, built from `nn.Linear` layers, learns the training images by heart: 99.7 % on train, about 40 % on new images. We monitored it, fixed it with regularisation (45.5 % on validation) and diagnosed four broken runs.
4. **A pretrained ResNet18**, frozen, with only one linear layer trained on top: **83.0 % on the test set**, against **67.3 %** for the best classical model (random forest on hand-made colour and texture features).
5. **Cost on a kiosk**: **53 ms per photo** on one CPU thread, a 45 MB file, 17 s of training on a laptop, no GPU.
6. **Answer**: yes, a neural network is worth it, but a **pretrained** one that we reuse, not one we train from zero. Before production: test it on real photos taken at the kiosks.

Four numbers to remember: 83.0 % (our choice, on test), 67.3 % (best classical), 53 ms (one photo, one thread), +11.4 points (the score inflation when the same painter is seen in training).

2. The basic ideas, explained simply

Read this chapter first. Each idea comes with an everyday picture, then what it means in our project. If you can tell these pictures, you can answer most questions.

The whole course in three sentences

A neural network is a machine with millions of small **knobs** (the weights). Training means turning the knobs, little by little, until the machine gives the right answers on the training paintings. The whole question of the project is: does it also give the right answers on paintings it has **never seen**?

A neuron, weights and a network

The picture. Think of a **jury member** who gives a score to a painting. They look at many small clues (a bit of red here, a dark corner there) and each clue counts more or less: that "how much it counts" is a **weight**. They add everything up and give a score. A neural network is many jury members in rows: the first row looks at the pixels, the next row listens to the first row, and so on. The last row gives one score per style.

In our project. Our MLP has 3 rows of jury members (512, then 256, then 5). The 5 final scores are the **logits**, one per style; the highest one is the answer.

The loss: how wrong we are

The picture. A **golf score**: the lower, the better. After each guess, the loss says how far the answer was from the truth. A confident wrong answer costs a lot; a confident right answer costs almost nothing.

In our project. We use the cross-entropy loss. A network that knows nothing gives 20 % to each of the 5 styles; its loss is then ln 5 = 1.609. This number is our **alarm**: a loss stuck at 1.609 means "I have learned nothing".

Gradient descent and the learning rate

The picture. You are on a **mountain in thick fog** and want to reach the valley (the lowest loss). You cannot see far, but you can feel the slope under your feet: that slope is the **gradient**. You take a step downhill, feel again, take another step. The **learning rate** is the size of your steps: baby steps and you need a year; giant leaps and you jump over the valley, or fall off a cliff.

In our project. Our learning-rate sweep tried 8 step sizes. 0.0001 was the best. 0.01 was a giant leap: the network broke (its neurons died, see step 4).

Backpropagation and the 5 lines of the training loop

The picture. After a guess, the **blame** is passed backwards, from the last row of the jury to the first: "you counted the red too much, you counted the dark corner too little". Each knob learns how it should move. The 5 lines of code are: empty the blame notebook (`zero_grad`), look at the paintings (forward), compute the golf score (loss), pass the blame backwards (`backward`), turn the knobs (`step`).

In our project. Forgetting to empty the notebook (`zero_grad`) is one of our four broken runs: the old blame piles up and the knobs go crazy.

Epoch and mini-batch

The picture. Reading a **textbook**: you read it page by page (a mini-batch = 64 paintings) and you adjust your understanding after each page. One full read of the book is one **epoch**. You may read the book 40 times.

In our project. Our MLP saw its 1 061 training paintings 40 to 60 times; each epoch takes less than a second for the MLP, and 15 to 30 seconds for the fine-tuned ResNet.

Train, validation, test

The picture. School: **homework** (train: you learn from it, with the answers), **mock exams** (validation: you use them to decide how to study, which settings to keep), and the **final exam** (test: taken once, at the very end). If you look at the final exam while studying, your grade means nothing.

In our project. 1 061 paintings to train, 219 to validate, 220 to test. All our choices were made on validation; the test was opened once, in notebook 06.

Overfitting

The picture. A student who **learns the answers of past exams by heart** instead of understanding the lesson: perfect on the past exams, lost on a new one. The bigger the memory compared with the number of examples, the easier it is to cheat this way.

In our project. Our MLP had 6.4 million knobs for 1 061 paintings (6 000 knobs per painting!): 99.7 % on its training paintings, 40 % on new ones.

ReLU, the activation

The picture. A **one-way valve** on a water pipe: if the signal is positive it goes through, if it is negative the valve closes and lets nothing through (max(0, z)). Without valves, stacking rows of jury members would be no better than one row: the valves let the network bend its decisions.

In our project. All our MLPs use ReLU. A neuron whose valve is closed for **every** painting is a **dead neuron**: like a burnt light bulb, it never lights up again and never learns.

Initialisation and the first loss

The picture. Before a concert you set the **volume** of each microphone: too loud and everything saturates, too quiet and nobody hears. He initialisation sets the starting volume of the weights so that the signal keeps the same strength from row to row.

In our project. With the usual setting, our MLP started too sure of itself (first loss 1.99 instead of 1.61): like a student answering loudly before reading the question. Turning down the last row (x 0.1) gave 1.59, a fair "I don't know yet".

Standardisation of the pixels

The picture. Before comparing students from two schools, you **put the grades on the same scale** (for example, how far each grade is from the class average). For pixels: subtract the average colour, divide by the spread, so values sit around 0 instead of 0 to 255.

In our project. We use the average and spread of the **training** paintings only. Giving raw values 0 to 255 is one of our broken runs: the first loss jumped to 16.3.

MLP versus CNN: why the MLP cannot see shapes

The picture. The MLP receives the painting as **3 072 puzzle pieces poured into a bag**: it gets every colour value, but not which piece was next to which. It cannot see a line, a face or a brush stroke. A CNN looks at the painting through a small **magnifying glass** that slides over the canvas: it sees local patterns (an edge, a texture) wherever they are.

In our project. This is why our MLP recognises colours (dark Baroque, light Ukiyo-e) but almost never Cubism (7 %), which is about shapes; the ResNet (a CNN) finds most Cubist paintings.

Convolution, filters and channels (session 7 slide)

The picture. A **filter** is a small **stencil**, for example 3 x 3 pixels, that slides over the image and asks everywhere the same question ("is there a vertical edge here?"). A colour image has 3 **channels** (red, green, blue), like 3 transparent sheets stacked; the stencil goes through all the sheets at once, so a 3 x 3 filter on a colour image has 3 x 3 x 3 = 27 weights. Each filter produces one new sheet (a **feature map**); 64 filters produce 64 sheets, so the next layer has 64 channels.

In our project. The number of parameters of a convolution layer is C_out x (C_in x k x k + 1), the +1 being the bias. Example: 16 filters 3 x 3 on a colour image = 16 x (27 + 1) = 448 weights, whatever the size of the image, because the **same stencil is reused everywhere**. Compare: the first layer of our MLP on a 32 x 32 image has 1 573 376 weights. The first layer of ResNet18 has 64 filters of 7 x 7 on 3 channels = 9 408 weights (no bias).

A pretrained network and transfer learning

The picture. Instead of teaching a **newborn** to see, we hire an **experienced art expert** who has already looked at 1.2 million photos (ImageNet). Their first layers know edges and colours, the middle ones textures, the last ones parts of objects. We reuse their eyes for our paintings: this is transfer learning.

In our project. ResNet18: 18 layers, 11.2 million weights, trained by others on ImageNet. We did not train it; we reused it.

Frozen features (linear probe) versus fine-tuning

The picture. Frozen: the expert describes each painting with 512 notes ("many soft edges, strong blue, flat areas...") and we only learn a small **grading sheet** that turns these notes into a style. **Fine-tuning**: we also retrain part of the expert's eyes on our paintings. With few paintings, the retrained expert starts to memorise them.

In our project. Frozen + linear head: 2 565 knobs to learn, 0.4 s of training, 88.1 % on validation. Fine-tuning the last block: 8.4 million knobs, minutes of training, 85.4 % on validation.

The tools against overfitting

The picture. Weight decay is a **tax on big weights**: the network cannot rely on a few huge knobs. **Dropout** is a football team that trains with **random players sent to the bench** each time, so nobody relies on one star. **Batch norm re-centres the grades** of each class so every teacher receives comparable numbers. **Flips** show the painting **in a mirror**: a new example for free. **Early stopping** takes the **cake out of the oven** when it is cooked, before it burns. A **cosine schedule** slows down smoothly, like **parking a car**.

In our project. Alone, each tool helped a little; together they lifted our MLP from 38.8 % to 45.5 % and cut the gap between train and validation from 0.61 to 0.12.

Vanishing gradient

The picture. The **whisper game**: the first person says a sentence, each person repeats it more quietly. After 8 people, the first ones in the chain hear almost nothing. With sigmoid activations, the blame passed backwards is multiplied by at most 1/4 at each layer.

In our project. With 8 sigmoid layers, the first layer received 6 x 10^-7 of the blame of the last one: it never learned. ReLU and He initialisation fixed it.

Data leakage

The picture. A teacher who **recognises a student's handwriting** gives a good grade for the wrong reason. If a painter's works are both in training and in the test, the model can recognise the painter's hand instead of the style, and the score looks better than it really is.

In our project. We measured it: the same model scores 11.4 points higher on known painters when it has seen other works of the same painter. That is why our split keeps each known painter in one part only.

Confidence interval and bootstrap

The picture. Imagine **re-running the final exam** with another random set of 220 paintings, thousands of times: the score would move up and down. The bootstrap simulates this by redrawing our 220 test paintings (with replacement) 5 000 times. The range that holds 95 % of the scores is the 95 % interval.

In our project. Our chosen model: 83.0 %, interval 78.6 to 88.6 %. A **paired** bootstrap compares two models on the same redrawn paintings: ResNet minus random forest = +16.4 points, interval 9.1 to 24.1, so it is not luck.

Latency and CPU threads

The picture. Latency is the waiting time of a visitor between the photo and the answer. **Threads** are like **cashiers** in a shop: more cashiers, shorter queue, but a small kiosk has few of them.

In our project. 53 ms per photo on one thread, 26 ms on 4. A blink of an eye lasts about 100 to 400 ms: the visitor will not notice the wait.

3. Step 1: the data and a fair split (notebook 01)

In simple words

Think of a **cooking contest**. If the jury has already tasted many dishes of one chef during training, they may recognise **the chef's touch** instead of judging the cuisine. So we put all the dishes of a known chef in only one pile: training, mock exam or final exam, never two.

We took 1 500 paintings (5 styles x 300), checked who painted them, cut them into 3 piles of the same mix of styles, and locked the final-exam pile until the very end.

We used 1 500 WikiArt paintings, 300 per style, and split them so that no known painter appears in two parts.

What we downloaded

The course script `make_wikiart_subset.py` streams the `huggan/wikiart` dataset (81 444 paintings, 27 styles) at a fixed revision. We chose 5 styles that look different: Impressionism, Baroque, Cubism, Ukiyo-e and Pop Art. Each image is a 224 x 224 centre crop, the input size of ResNet.

- **Why `--strategy spread --seed 3`.** With the default plan, 1 402 of the 1 500 images had the label "Unknown Artist". We compared plans with a dry run (labels only, no results) and picked one with **618 images by a known painter** (32 painters). It streams 8.4 GB once instead of 4.2 GB. We needed known painters to test painter leakage.
- **`--max-per-artist 50`.** One painter gives at most 50 images; otherwise a style could be mostly one painter and the model would learn the painter.
- **Licence.** WikiArt images keep their authors' rights: no image in Git, no painting in the committed notebooks (`SHOW_ARTWORKS=0`), the data folder is ignored.

The split (make_split in data.py)

- 1 061 train, 219 validation, 220 test.
- **Stratified**: every part has about the same number of each style (about 43 per style in validation and test).
- **Grouped by painter** (`StratifiedGroupKFold`): all images of a known painter go to the same part. Result: **0 known painters in two parts**, against **23 of 32** with a random split.
- "Unknown Artist" images are each their own group: we cannot group painters we do not know (a limit).
- The split is saved in `reports/split.csv`; the test set stays locked until notebook 06.

What we found in the data

- Classes are balanced, so chance is 20 % (22.8 % on validation, because Impressionism has 50 images there).
- Known painters per style: Impressionism 300 of 300 (11 painters; Monet, Renoir and Pissarro at 50 each), Cubism 155, Baroque 88, Ukiyo-e 61, Pop Art only 14 (Andy Warhol).
- Simple colour numbers separate little: Baroque is darker (mean brightness 0.45 against 0.57 to 0.64); saturation separates almost nothing.
- No near copies: 242 pairs have close fingerprints (average hash), but the highest correlation is 0.93, below our 0.95 threshold.

4. Step 2: the MLP and its two checks (notebook 02)

In simple words

Our MLP is a student who looks at each painting as **puzzle pieces in a bag** (3 072 colour values, no idea which piece was next to which).

Check 1: before any lesson, a fair student should say "I don't know, 20 % for each style" (loss 1.609). Ours answered loudly and wrongly (1.99), so we turned down its last row: 1.59.

Check 2: can it learn 32 flashcards by heart? Yes, 100 %: its brain and our training loop work.

Result: it learned its 1 061 training paintings by heart (99.7 %) but recognised only 40 % of new ones. It **memorised past exams** instead of understanding the lesson. Bigger images (64 px) did not help: more puzzle pieces in the bag, still no shapes.

Our first network learns its training images by heart (99.7 %) but reaches only 40.6 % on validation: strong overfitting.

The model

`make_mlp` in `models.py`, built like in the course: `nn.Sequential` of `nn.Flatten`, `nn.Linear` and `nn.ReLU`. Input 64 x 64 x 3 = 12 288 pixels, hidden layers of 512 and 256 neurons, 5 outputs (logits; `nn.CrossEntropyLoss` applies the softmax). **6 424 581 parameters**, checked against the session 3 formula: a layer with n_out neurons and n_in inputs has n_out x (n_in + 1) parameters. That is 6 055 parameters per training image.

Standardisation. Pixels in [0, 1], then (x - mean) / std per colour channel, with the mean and std of the **training images only**.

Check 1: the first loss

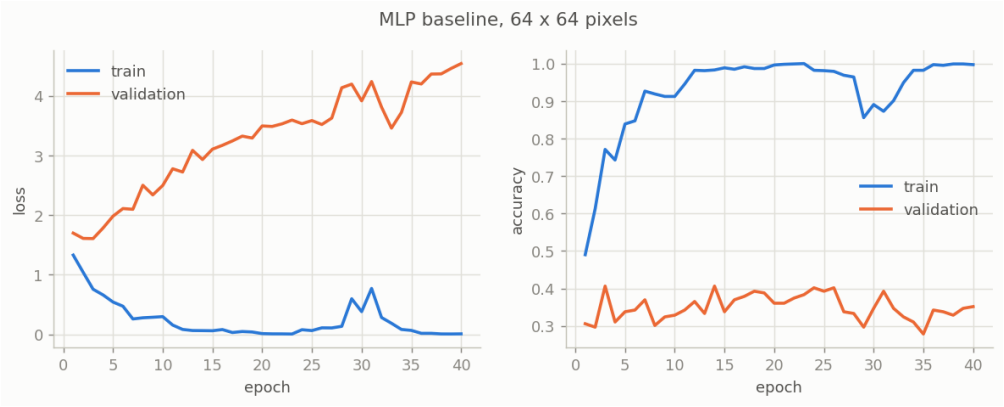
A model that knows nothing gives 1/5 to each style, so the cross-entropy is ln 5 = 1.609. With the usual He initialisation the first loss was **1.990**: the first logits were too big (std 1.34), so the model started confident and wrong. Multiplying the last layer's weights by 0.1 (`last_scale=0.1`) gave **1.591**, almost ln 5. We kept it for all MLPs.

Check 2: overfit one batch

Train on 32 images only for 150 steps: the loss goes from 1.537 to 0.0000 and the accuracy to 100 %. The model and the 5-line loop (zero_grad, forward, loss, backward, step) work.

Baseline and image size

- Adam, learning rate 0.001, batches of 64, 40 epochs, no regularisation.
- Validation accuracy best at **epoch 3: 40.6 %**, then down to 35.2 % at epoch 40; training accuracy up to **99.7 %**; validation loss rising from 1.606 to above 4.
- Image size (3 seeds each): 16 px 39.4 %, 32 px 39.0 %, 64 px 37.1 %. More pixels do not help an MLP: it sees a flat list of numbers and does not know which pixels are neighbours. We kept 32 x 32 for the next steps (1.7 M parameters).



MLP baseline: training loss falls to 0 while validation loss rises (overfitting).

5. Step 3: monitoring and regularisation (notebook 03)

In simple words

A **doctor** does not only take the temperature (the loss): they also check the pulse, the blood pressure, the reflexes. We checked three things inside the network after each epoch:

1) does the **blame signal reach every floor** of the building (gradient per layer)? Yes. 2) how many **light bulbs are burnt out** (dead neurons)? Only 4 %. 3) how **loud** are the weights (weight norm)? Growing while it memorises. Diagnosis: the patient is healthy, it just **memorises too much**.

Then we tried **8 step sizes** on the foggy mountain (learning rates) and kept 0.0001. Finally we tried the anti-memorising tools one by one (tax on big weights, players on the bench, mirror images, cake out of the oven in time...). Each one helped a little; **all together** they took the student from 38.8 % to 45.5 %. But the student still only sees colours: it finds 7 % of the Cubist paintings.

The network was healthy but overfitting; all the tools of session 6 together lifted it from 38.8 % to 45.5 % on validation.

What we monitored

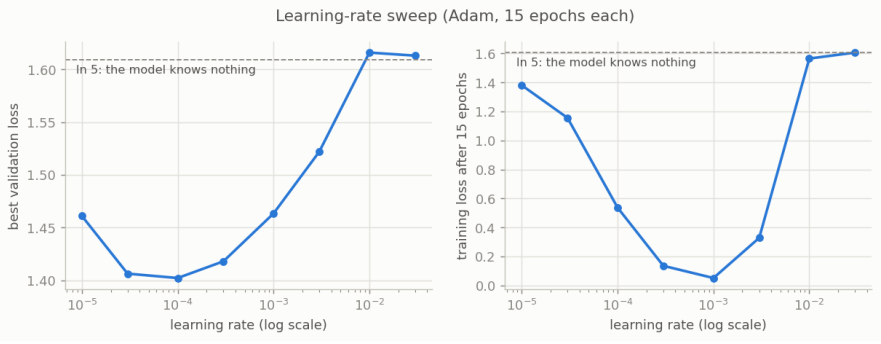
After each epoch, on a fixed batch of 256 training images (`make_monitor` in `diagnostics.py`, called through `fit(on_epoch_end=...)`):

- **Gradient norm of each layer**: at epoch 1, layer 1 got 1.63 and layer 3 got 30.0: smaller but flowing. At epoch 40 all norms are below 0.1: nothing left to learn, the model knows its training images by heart.
- **Dead neurons** (a ReLU that gives 0 for every image): 0 % in layer 1, about 4 % in layer 2.
- **Weight norm**: grows from 39.6 to 47.8 while the network memorises.

Conclusion: the training is healthy; the problem is overfitting, not a bug.

Learning-rate sweep (Adam, 8 values, 15 epochs each)

- 0.00001: too slow (training loss still 1.38).
- **0.0001: best validation loss (1.402)**, chosen.
- 0.001 (Adam's default, used in step 2): learns fastest, overfits after 2 epochs (1.463).
- 0.01 and 0.03: loss stuck at ln 5, accuracy about 19 %: the network is broken (dead neurons, see step 4).



Learning-rate sweep: a U-shaped validation loss, best at 0.0001.

The tools against overfitting (lr 0.0001, 3 seeds each)

Tool	Val accuracy	Gap train - val	Note
Baseline (lr 0.0001)	38.8 %	0.61	
Weight decay (AdamW, wd 5)	39.7 %	0.59	weight norm 40.5 to 29.3

Tool	Val accuracy	Gap train - val	Note
Dropout (0.5 hidden, 0.2 input)	41.7 %	0.38	
Batch norm	39.3 %	0.61	lowest val loss of single tools (1.350)
Flips (augmentation)	42.8 %	0.54	best single tool
Early stopping (patience 5)	41.9 %	0.27	13 epochs, 1.8 s, cheapest
All together + cosine, 60 epochs	**45.5 %** (std 0.7)	**0.12**	our best MLP

Alone, each tool is near the noise; together they add up. Why wd = 5 looks big: AdamW multiplies each weight by (1 - lr x wd) at each step, so with lr = 0.0001 it is a 0.05 % shrink per step.

Errors of the best MLP (validation): Baroque recall 69 %, Impressionism 62 %, Ukiyo-e 57 %, Pop Art 33 %, **Cubism 7 %** (3 of 42), mostly taken for Pop Art. The MLP recognises **colours** (dark Baroque, light Ukiyo-e), not **shapes** (Cubism is about geometric facets).

6. Step 4: the training doctor (notebook 04)

In simple words

We made the network sick on purpose, four times, to learn to recognise each illness:

1) **Learning rate too big**: giant leaps on the foggy mountain. The network jumps over the valley; its light bulbs burn out (99.6 % dead in layer 2) and it answers "Baroque" for almost everything. With an even bigger step, the numbers grow like an **avalanche** until the computer cannot count them (nan).

2) **zero_grad forgotten**: a **shopping cart never emptied**. Each trip adds new items to the old ones, until the cart is huge (the blame is 26 times bigger after 2 epochs) and the steps go wild.

3) **Raw pixels (0 to 255)**: **shouting** at someone who expects whispers. The first loss jumps to 16.3 instead of 1.6.

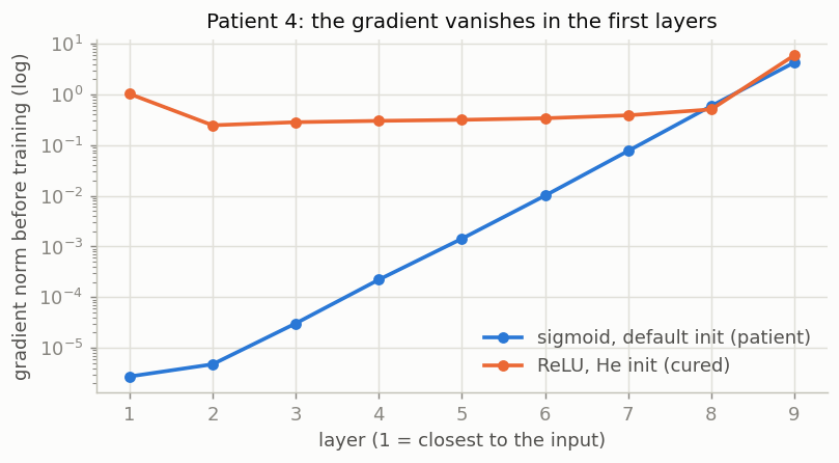
4) **8 sigmoid layers**: the **whisper game**. The blame fades at each layer; the first layer gets almost nothing (6 x 10^-7) and never learns.

Key lesson: illnesses 1 and 4 have the **same fever** (loss stuck at 1.609). Only the other checks (burnt bulbs, signal per floor) tell them apart.

We broke the training four times on purpose; each failure has its own signature, and two of them look the same in the loss.

Method for each patient: **symptom** (what we measure), **diagnosis** (the cause, with numbers), **cure** (change one thing, measure again).

Patient	Symptom (measured)	Cause	Cure and result
1. Learning rate too big (Adam 0.01)	loss stuck near ln 5; 212 of 219 images predicted Baroque; **99.6 % of the neurons of layer 2 dead**. With SGD at lr 1: loss 1.6, 17.9, 3 140... then nan at mini-batch 8	Adam moves each weight by about lr per step; the ReLUs of layer 2 become negative for every image and die (gradient 0 forever)	lr 0.0001: 43.8 % best val, 0.4 % dead
2. zero_grad() forgotten	loss jumps to 4.94, then one class only (19.2 %)	backward() ADDS to .grad: step k uses the sum of k gradients; the norm is 26 times bigger after 2 epochs	add zero_grad(): smooth loss, 41.1 %
3. Raw pixels (0 to 255)	first loss **16.3** instead of 1.6; SGD loss 322 000 then 306 million; Adam noisy (35.2 %)	inputs dozens of times too big: huge logits and gradients	standardise: first loss 1.610, 43.8 %
4. 8 sigmoid layers, default init	loss stuck at **1.609 = ln 5**, 19.2 %	**vanishing gradient**: layer 1 gets 2.7e-6, layer 9 gets 4.3 (ratio 6e-7); the sigmoid derivative is at most 1/4	ReLU + He init: ratio 0.17, 39.3 %



Patient 4: the gradient vanishes layer after layer with sigmoids; ReLU + He init fixes it.

Lessons to say at the oral

- Patients 1 and 4 show the **same symptom** (loss stuck at ln 5) for **two different causes**: only the monitor tells them apart (dead neurons for one, gradient per layer for the other).
- Patient 3 is **silent**: Adam does not crash, it just learns worse. The first-loss check catches it at once.
- Our checklist: first loss about ln C; one batch learned by heart; the gradient reaches every layer; few dead neurons; always plot train AND validation curves.
- Course table (session 6): nan means lr too high; loss flat at ln C means a bug or dead units; train down and val up means overfitting; both high means underfitting.

7. Step 5: pretrained ResNet18 (notebook 05)

In simple words

Instead of a newborn student, we hire an **experienced art expert** (ResNet18, who has seen 1.2 million photos). The expert looks at each painting through a **sliding magnifying glass** and writes **512 notes** about edges, textures and patterns.

Frozen + linear head: we keep the expert as they are and only learn a small **grading sheet** that turns the 512 notes into a style: 88.1 % on validation, twice our MLP, with 0.4 s of training. **Fine-tuning**: we also retrain the last part of the expert's eyes; with only 1 061 paintings, the expert starts memorising: 85.4 %.

Two silent mistakes: 1) forgetting `model.eval()` is like grading each painting **only against the others in the same box**: in a box full of Baroque paintings, "dark" looks normal, so the style disappears (12.8 %). 2) forgetting the ImageNet normalisation is like showing the expert the paintings **under a strange light** (81.7 %).

The **handwriting test**: when the model has seen other works of the same painter, it scores 11.4 points higher on that painter. It partly recognises the painter's hand, not only the style. Our split avoids this.

A frozen ResNet18 with one trained linear layer reaches 88.1 % on validation, almost twice our best MLP, with only 2 565 trained parameters.

Why a CNN, and why pretrained

A CNN slides small filters over the image, so it finds local patterns (edges, textures, brush strokes) anywhere and builds bigger shapes layer after layer. Training one from zero needs far more than 1 061 images, so we reuse **ResNet18** trained on **ImageNet** (1.2 million photos, 1 000 classes): **transfer learning**. ResNet18 has 18 layers, 11.2 million parameters, and shortcut (residual) connections that let the gradient flow through deep networks.

Ours vs library. torchvision gives ResNet18 and its weights. Ours: the head, the freezing (`ResNetClassifier` in `models.py`), the ImageNet normalisation module, the feature cache, the training loop and all the checks.

Method A: frozen features + head

- Each painting goes once through the frozen ResNet (eval mode) and becomes **512 numbers** (15.5 ms per image in batches of 64).
- **Linear probe**, one `nn.Linear(512, 5)`, 2 565 parameters: **88.1 %** on validation (the same for 3 seeds: fixed features give the same solution), trained in **0.4 s**.
- Small **MLP head** (512, 256, 5, dropout): 89.2 % (std 1.1), the same within the noise. Our MLP is useful again, on top of good features.

Method B: fine-tuning layer4

The last block (8.4 million parameters, 75 % of the network) also learns, with random crops and flips, AdamW (lr 0.0001 for layer4, 0.001 for the head), a cosine schedule and early stopping. Result **85.4 %** (seeds: 85.8, 84.9, 85.4), **below** the linear probe: training accuracy 99.9 %, best epoch 2 to 7, then overfitting; 1.7 to 3.6 minutes per run. The frozen blocks keep their batch norm in eval mode (`ResNetClassifier.train()` forces it).

Extra pilot runs (single runs, not in the repository): a smaller lr for layer4 (0.00003) gave 83.6 %; starting from the trained linear probe (LP-FT) gave 86.8 %. Both still below the linear probe.

Two silent bugs, measured

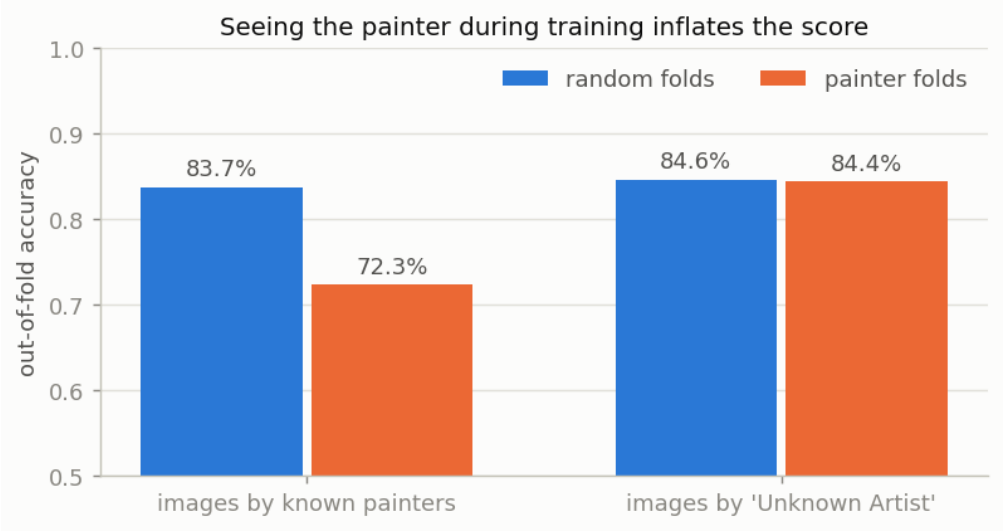
- Forgetting `model.eval()` during extraction: **12.8 %**, below chance. In train mode batch norm uses each batch's statistics; our batches held mostly one style, so it erased exactly the style.
- Forgetting the ImageNet normalisation: **81.7 %** (minus 6.4 points).

Painter leakage, measured fairly

5-fold cross-validation on train + validation (1 280 paintings, test still locked), linear probe, 3 random cuts, the **same images scored** in both cases:

Folds	Images by known painters (536)	Images by "Unknown Artist" (744)
Random (the painter is usually seen in training)	83.7 %	84.6 %
By painter (the painter was never seen)	72.3 %	84.4 %

Seeing the painter adds **11.4 points**; the control group does not move (standard deviations below 1 point). The model partly recognises **painters**, so a random split would flatter the score. This is why our split groups painters.



Painter leakage: same images, same model; only the known-painter group changes.

Errors of the linear probe (validation): 26 mistakes out of 219 (the MLP made 118). Ukiyo-e recall 100 %, Baroque 93 %, Impressionism 90 %, Pop Art 86 %, Cubism 71 %. Main confusion: Cubism and Pop Art, both ways (8 and 5): both use flat strong colours and hard edges.

8. Step 6: final test, CPU cost and recommendation (notebook 06)

In simple words

The final exam, taken once. Every candidate answers the same 220 paintings that nobody saw before.

The classical candidates use a **checklist written by a human** (colour histograms, directions of the edges): the best one, a random forest (500 decision trees that vote), gets 67.3 %. Our expert with a grading sheet gets 83.0 %, our MLP 41.4 %.

To know if the gap is luck, we **re-run the exam 5 000 times** in simulation with redrawn paintings (bootstrap): the expert stays ahead by 9 to 24 points. Not luck.

Cost: the expert answers in 53 ms on one CPU thread, **faster than a blink of an eye** (100 to 400 ms), fits in 45 MB and learns its grading sheet in 17 s on a laptop. So the director's answer: yes, hire the pretrained expert; do not train a network from zero.

On 220 paintings never seen before, the frozen ResNet18 + linear head reaches 83.0 %, 16 points above the best classical model, in 53 ms per photo on one CPU thread.

Classical models (settings chosen on validation)

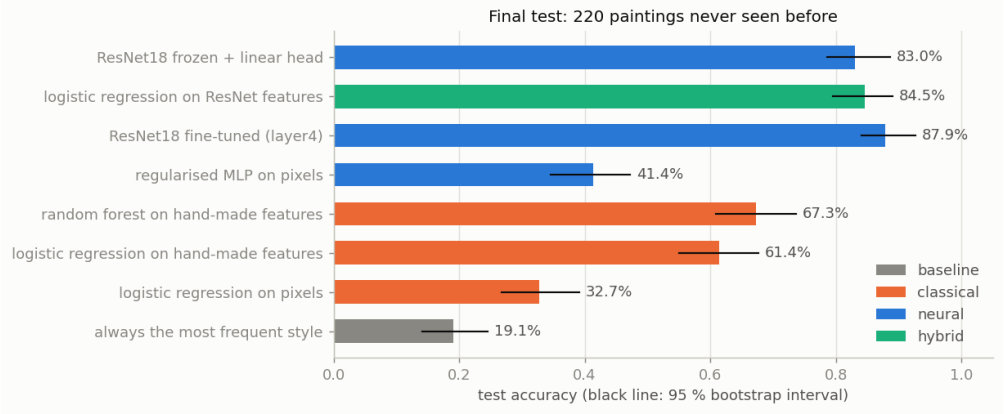
- Logistic regression on pixels (32 px): 40.6 % on validation.
- **Hand-made features** (`baselines.py`, 102 numbers): HSV colour histogram 8 hues x 3 saturations x 3 brightness levels (72), colour statistics (10), histogram of edge directions weighted by edge strength at two scales (20). Logistic regression 61.2 %, **random forest** (500 trees) 63.5 % on validation.
- Logistic regression on the ResNet features (a "hybrid"): 88.1 %, the same as our linear probe.

Reproducibility check. We trained the 3 neural models again with `recipes.py`: all 9 runs give **exactly** the validation accuracy of notebooks 03 and 05.

The final test, measured once

For 3-seed models: mean accuracy, interval from seed 0. Times: median over 50 photos, one at a time, including resizing and features.

Model	Test accuracy (95 % interval)	1 photo, 1 thread	File
Most frequent style	19.1 %		
Logistic regression, pixels	32.7 % (26.8 to 39.1)	0.35 ms	0.1 MB
Logistic regression, hand-made features	61.4 % (55.0 to 67.7)	1.5 ms	0.006 MB
Random forest, hand-made features	67.3 % (60.9 to 73.6)	24 ms	26 MB
Regularised MLP, pixels (ours)	41.4 % (34.5 to 47.3)	0.36 ms	6.8 MB
ResNet18 frozen + logistic regression	84.5 % (79.5 to 89.1)	53 ms	45 MB
ResNet18 frozen + linear head (ours, chosen)	**83.0 % (78.6 to 88.6)**	**53 ms**	**45 MB**
ResNet18 fine-tuned (layer4)	87.9 % (84.1 to 92.7)	53 ms	45 MB



Final test: 220 paintings never seen before, with 95 % bootstrap intervals.

Are the gaps real? Paired bootstrap (same 220 paintings, 5 000 redraws)

- Frozen ResNet vs random forest: **+16.4 points** (interval 9.1 to 24.1), never not better.
- Frozen ResNet vs MLP: +42.7 points; random forest vs MLP: +26.4 points.
- Frozen vs fine-tuned: -5.0 points on test (interval -9.1 to -0.9).

Frozen or fine-tuned? Validation and test disagree

Validation: frozen 88.1 %, fine-tuned 85.4 %. Test: frozen 83.0 %, fine-tuned 87.9 %. With about 220 images per set, a score moves by about 5 points by chance. We chose the frozen model on validation **before** the test and keep it: choosing a model by its test score would make that score too optimistic. Fine-tuning is the first thing to retry with more data.

Speed and size

ResNet18: 53 ms per photo on 1 thread, 37 ms on 2, 26 ms on 4. A kiosk 4 times slower: about 0.2 s. 45 MB (11 million numbers in 32 bits). Training: 17 s (features of 1 061 images + 0.4 s head); fine-tuning 153 s. No GPU.

Recommendation to the director

Deploy **ResNet18 frozen + a linear head**:

1. **Accurate**: one visitor in six gets a wrong style, against one in three with the best classical model.
2. **Fast enough** for a modest CPU.
3. **Cheap for a small team**: the ResNet is used as downloaded; when the collection changes, compute the new features and retrain the head in seconds.

Not recommended: our MLP on pixels (41 %), a classical model alone (61 to 67 %), fine-tuning for now (minutes instead of seconds, not proven better). Before production: test on a few hundred real kiosk photos, add styles and paintings, measure the real kiosk hardware.

9. Key numbers to know by heart

What	Number
Data	1 500 paintings, 5 styles x 300, 224 x 224 px
Split (train / val / test)	1 061 / 219 / 220, grouped by painter
Known painters in two parts	0 (random split: 23 of 32)
Chance level	20 % (22.8 % on validation)
First loss: ln 5 / He init / last layer x 0.1	1.609 / 1.990 / 1.591
MLP baseline (64 px, 6.4 M parameters)	40.6 % val at epoch 3, 99.7 % train
Best learning rate (sweep)	0.0001
Regularised MLP (32 px, all tools)	45.5 % val (std 0.7), gap 0.12
Linear probe on ResNet18 features	88.1 % val, 2 565 trained parameters, 0.4 s
Fine-tuned ResNet18 (layer4)	85.4 % val, 8.4 M trained parameters
Bug: no model.eval() / no ImageNet normalisation	12.8 % / 81.7 %
Painter leakage	+11.4 points (83.7 % vs 72.3 %), control +0.2
Vanishing gradient (8 sigmoid layers)	layer 1 / layer 9 = 6e-7
Test: frozen ResNet18 + linear head (chosen)	**83.0 % (78.6 to 88.6)**
Test: fine-tuned ResNet18	87.9 %
Test: best classical (random forest, hand-made)	67.3 %
Test: regularised MLP	41.4 %
Gap ResNet vs random forest (paired)	+16.4 points (9.1 to 24.1)
Speed ResNet18, one photo	53 ms (1 thread), 26 ms (4 threads)
Size / training time	45 MB / 17 s, no GPU

10. Concepts in simple words

Term	Simple explanation
MLP (feed-forward network)	Layers of neurons; each neuron is connected to all values of the layer before. Input = a flat list of pixels.
nn.Linear	One fully connected layer: output = W x + b, weights of shape (out, in).
ReLU	max(0, z). Lets the gradient pass (derivative 1) for positive inputs.
Logits, softmax, cross-entropy	Logits = raw scores; softmax turns them into probabilities; cross-entropy = -ln(probability of the true class).
ln C	The loss of a model that knows nothing (1/C for each class). ln 5 = 1.609.
He (Kaiming) initialisation	Random weights scaled so the signal keeps the same size from layer to layer with ReLU.
Standardisation	(x - mean) / std, with the statistics of the training set only.
Epoch, mini-batch	Epoch = one pass over all training images; mini-batch = the images used for one update (64 here).
The 5 lines of the loop	zero_grad, forward, loss, backward, step.
Adam / AdamW	Optimisers that adapt the step of each weight; AdamW applies weight decay separately.
Learning rate	Size of each step. Too small = slow; too big = unstable, nan or dead neurons.
Overfitting / underfitting	Learns the training images by heart and fails on new ones / too weak, train and val both bad.
Weight decay (L2)	Each step shrinks the weights a little, so the model cannot rely on a few huge weights.
Dropout	During training, switch off each neuron with probability p; off at test time.
Batch normalisation	Re-centre and re-scale each layer on every mini-batch; at test time it uses running statistics (model.eval()).
Early stopping	Stop when the validation loss stops improving; keep the best epoch's weights.
Cosine schedule	The learning rate goes down smoothly to 0 along half a cosine.
Data augmentation	New training images made by flipping or cropping: a form of extra data.
Vanishing gradient	The gradient becomes tiny in the first layers, so they stop learning.
Dead ReLU	A neuron that outputs 0 for every input: gradient 0, it never learns again.
CNN, convolution	Small filters slid over the image; they find local patterns anywhere.
ResNet18, ImageNet	An 18-layer CNN with shortcut connections, pretrained on 1.2 million photos of 1 000 kinds of objects.
Transfer learning	Reuse a network trained on another task.
Linear probe / fine-tuning	Frozen network gives 512 numbers, train one linear layer on top / also retrain some pretrained layers with a small learning rate.
Stratified / grouped split	Same share of each class in each part / all items of a group (a painter) in the same part.
Data leakage	Information from the evaluation data gets into training, so the score is too good.
k-fold cross-validation	Cut the data in k groups, train on k-1, test on the last, k times.
Bootstrap / paired bootstrap	Redraw the test images with replacement thousands of times to see how much a score (or a difference between two models) moves.
Precision, recall, F1, macro F1	Recall = share of a class found; precision = share of predictions right; F1 = their harmonic mean; macro F1 = mean over classes.

Term	Simple explanation
Confusion matrix	Rows = true class, columns = predicted class.
Latency, median, p95	Time for one photo; median = typical; p95 = 95 % of photos are faster.
Random forest / logistic regression	Many decision trees that vote / a linear model with a softmax (the classical twin of our linear probe).

11. Questions you may get, with answers

Answer with the number first, then the reason in one sentence.

About the data

Why these 5 styles?

They look different (dark Baroque, flat Ukiyo-e prints, colourful Pop Art), so a model can succeed, and the course suggested 5 to 8 classes. 300 each keeps the training on a CPU.

Why did you not use the default download plan?

It gave only 98 images with a known painter. The spread plan with seed 3 gave 618, which we needed to test painter leakage. We chose it from the labels only, before any result.

How do you know the model learns the style and not the painter?

Three things: at most 50 images per painter, a split grouped by painter (0 shared painters), and a measured test: seeing the painter adds 11.4 points, so grouping matters, and we did it.

What about "Unknown Artist" (59 % of the images)?

We cannot group painters we do not know, so some leakage may remain there. It is a stated limit.

Why no images in Git?

The WikiArt licence is unknown and the course script says not to commit or publish them. We commit the script and the exact command instead.

About the MLP

Is it really built "from primitives"?

Yes: nn.Sequential of nn.Linear and activations, our own initialisation, our own training loop (the 5 lines), no high-level training library. Only the pretrained ResNet comes from torchvision, as the brief asks.

Why is the MLP so weak (41 to 46 %)?

It sees a flat list of pixels with no idea which pixels are neighbours, so no shapes or textures; and 1.7 to 6.4 M parameters for 1 061 images, so it memorises.

Why did you scale the last layer by 0.1?

With standard He init the first loss was 1.99, not ln 5 = 1.61: the model started over-confident. Smaller last weights give near-zero logits, so a fair start (1.591).

Why is a random forest better than your MLP?

Its 102 hand-made features already describe colour and texture; the MLP must find them alone in 3 072 raw pixels with only 1 061 images.

Weight decay = 5, isn't that huge?

With AdamW the shrink per step is lr x wd = 0.0001 x 5 = 0.05 %. The weight norm went from 40.5 to 29.3, so it worked as intended.

About training and diagnosis

How do you tell a learning rate that is too big from a vanishing gradient? Both stick at ln 5.

Look inside: the big learning rate killed 99.6 % of the neurons of layer 2; the vanishing gradient shows layer 1 getting a millionth of layer 9's gradient.

What would you check first on a new run?

First loss near ln C, overfit one batch, gradient per layer, dead neurons, both curves.

Why measure train metrics in eval mode?

So that dropout and batch norm behave the same on train and validation: the gap is then honest.

Why 3 seeds?

On 219 images one run is noisy (a few points); 3 seeds give a mean and a standard deviation.

About ResNet

Why not train a CNN from zero?

1 061 images are far too few for millions of weights; a pretrained network already knows edges, textures and shapes from 1.2 million photos.

Why ResNet18 and not a bigger model (ResNet50, ViT, CLIP)?

The brief asks for a ResNet and the kiosk CPU is small: ResNet18 is the smallest standard ResNet (53 ms per photo). We would only try a bigger one if accuracy were not enough.

Why did fine-tuning not win on validation?

8.4 M weights for 1 061 images: it overfits (99.9 % on train) even with augmentation. Frozen features plus 2 565 trained weights generalise better here.

Why does the linear probe give the same 88.1 % for 3 seeds?

The features are fixed and the problem is nearly convex, so every seed finds the same solution.

What does the 12.8 % bug teach?

Always call model.eval(): in train mode batch norm uses each batch's statistics, which erased the style because our batches held mostly one style.

About the results and the recommendation

Validation says frozen is better, test says fine-tuned is better. Which is right?

Neither can be proven: with about 220 images a score moves about 5 points by chance. We chose on validation before the test, which keeps the test honest. Fine-tuning is the first thing to retry with more data.

Why not pick the fine-tuned model, since it won the test?

Choosing by test score turns the test into a second validation set and makes the reported score too optimistic.

Is 83 % good enough for a museum?

One visitor in six gets a wrong style, against one in three for the best classical model. Good for a guide that tells the story of a movement; it must be confirmed on real kiosk photos.

Is 53 ms realistic on a kiosk?

Measured on one thread of a laptop, one photo at a time. A kiosk 4 times slower would still answer in about 0.2 s.

So is a neural network worth it?

Yes, a pretrained one: +16.4 points over the best classical model (interval 9.1 to 24.1), a cost a kiosk can afford, no GPU, retrained in seconds.

What about part 2 (questions in natural language)?

The written brief lists three deliverables and part 2 is not one of them, so we put our time on those and on the recommendation. Session 8 shows the next step (milestone m4): encode the visitors' questions and descriptions of the artworks with sentence embeddings (all-MiniLM-L6-v2), retrieve the closest artwork by cosine similarity, and compare with a classical TF-IDF search. Our WikiArt images have no text descriptions, so we would first have to write them.

About the project

How can someone reproduce your results?

pip install -r requirements.txt, bash scripts/get_data.sh, python -m pytest, python scripts/run_notebooks.py. Fixed seeds and a pinned dataset revision; notebook 06 re-trains the models and gets exactly the same validation scores.

How is the work organised?

A Python package (src/museum_guide), 6 notebooks paired with .py sources (clean Git diffs), 31 automatic tests, and every number saved in reports/results/*.json.

12. Limits to admit, and next steps

Saying the limits yourself, before the jury does, is a strength: each one comes with a next step.

Limit	Why it matters	Next step
Small test set (220 images)	A score moves about 5 points by chance; frozen vs fine-tuned cannot be separated	More paintings, or repeated cross-validation
Clean scans, not visitor photos	Real photos have angles, reflections, frames, bad light	Test a few hundred real kiosk photos before launch
"Unknown Artist" = 59 % of images	Their painters cannot be grouped; some leakage may remain	Better artist labels, or scores per painter
Only 5 styles, 300 each	Similar styles (Post-Impressionism) will be harder	Add styles and images, check per class
Speed measured on a laptop	A real kiosk CPU is slower	Benchmark on the real hardware; 8-bit model (about 4 times smaller, not tested)
Centre crop 224 x 224	The borders of the painting are cut	Try the whole painting
Fine-tuning tested in one setting	Other settings may beat the frozen model	Retry with more data, LP-FT, unfreeze layer3
Part 2 (milestone m4) not done	Not in the written list of deliverables	Sentence embeddings vs TF-IDF on artwork descriptions (TP08 part D)

13. Git history and how to reproduce

About 20 small commits, step by step, with four milestone tags (m1, m2, m3, v1.0); nothing was committed in one big block, and no image of a painting is in the repository.

Step	Commits	Tag
1. Setup	README, requirements, gitignore; the data script and the exact command	
2. Data + first MLP	data loading and split by painter (+ tests); notebook 01; MLP, training loop and two sanity checks (+ tests); notebook 02	m1
3. Monitoring + diagnosis	monitoring tools (+ tests); notebooks 01/02 refreshed without timestamps; notebook 03; notebook 04 + README diagnosis	m2 (asked by session 6)
4. ResNet18	ResNet wrapper, frozen blocks, feature cache (+ tests); notebook 05	m3 (asked by session 7)
5. Final	classical baselines, bootstrap, benchmark (+ tests); notebook 06; README results and recommendation	v1.0
6. Defence material	the presentation (public copy) and this study guide in docs/	

Repository map. `src/museum_guide/`: `data.py` (split by painter), `models.py` (MLP, ResNet wrapper), `training.py` (course loop: train_one_epoch, evaluate, fit with early stopping), `diagnostics.py` (first loss, one-batch overfit, gradient norms, dead neurons, lr sweep), `recipes.py` (the exact settings), `features.py`, `baselines.py`, `benchmark.py`, `evaluation.py` (confusion matrix, bootstrap). `notebooks/` 01 to 06 (.py source + .ipynb with outputs). `tests/`: 31 tests. `reports/`: figures, split, every number as JSON.

Reproduce.

1. `pip install -r requirements.txt`
2. `bash scripts/get_data.sh` (about 8.4 GB streamed once, 27 MB written)
3. `python -m pytest`
4. `python scripts/run_notebooks.py` (`FAST_RUN=1` for a quick check; about 45 minutes in full on a laptop CPU)

Repository: github.com/BenjaminAHS/neural-network-project

14. The 10-minute talk, slide by slide

About 55 seconds per slide; the speaker notes in the .pptx hold the full text in simple English.

#	Slide	Time	The one sentence to land
1	Is a neural network worth it on a museum kiosk?	0:30	The kiosk question and our five styles.
2	Our answer: yes, a pretrained network	1:00	83.0 % vs 67.3 % vs 41.4 %, at 53 ms per photo.
3	A fair test: one painter, one part	0:50	0 known painters in two parts; the test is used once.
4	Our MLP learns the paintings by heart	1:00	Two checks pass, then 99.7 % train vs 40.6 % validation.
5	Healthy training, but the MLP overfits	1:00	All tools together: 45.5 %; but Cubism 7 %: colours, not shapes.
6	Four broken runs, four signatures	1:00	Same symptom, two causes: the monitor tells them apart.
7	A pretrained ResNet18 sees shapes	1:00	88.1 % with 2 565 trained weights; fine-tuning overfits.
8	Seeing the painter inflates the score	0:50	+11.4 points on known painters, no change on the control group.
9	Final test: the pretrained network wins	1:00	+16.4 points over the random forest, interval 9.1 to 24.1.
10	Fast and cheap enough for a kiosk	0:40	53 ms, 45 MB, 17 s of training, no GPU.
11	Deploy a frozen ResNet18 + linear head	1:00	Why, not now, before launch; then thank you.

Slides 12 to 15 are backups for the questions: validation vs test, why you can trust the numbers, the training recipes, and where the chosen model fails.